

Bloc 2 TD 1 - Les Triggers

Contexte	2
Mise en place de la base HHB-Direct	3
Exercice d'application : numéro d'agence automatique	4
Exercices	5
Exercice 1 : mise à jour des soldes lors d'un virement	5
Exercice 2 : complétion automatique d'un nouveau client	7
Exercice 3 : contrôle des périodes de rémunération	9
Conclusion	10

Contexte

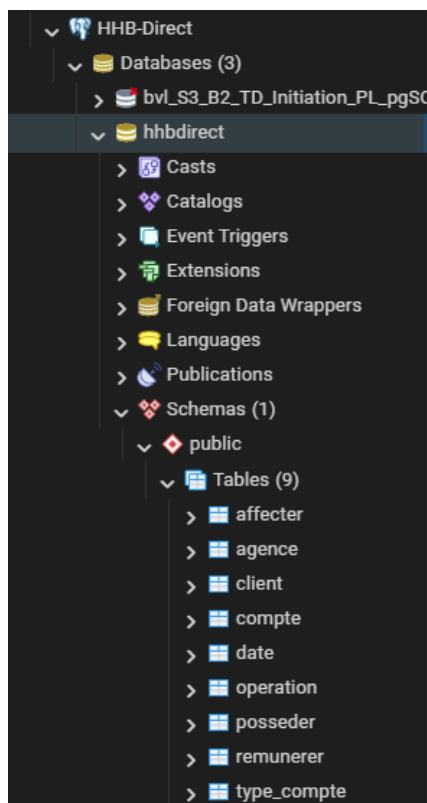
Ce TD nous fait découvrir les **triggers** (ou **déclencheurs**). Un trigger est une fonction stockée dans PostgreSQL qui s'exécute **automatiquement** quand un évènement arrive sur une table : un **INSERT**, un **UPDATE** ou un **DELETE**. On peut le déclencher **avant (BEFORE)** ou **après (AFTER)** l'opération, et pour chaque ligne concernée (FOR EACH ROW).

On travaille sur la base **HHB-Direct**, la base d'une banque en ligne (clients, comptes, opérations). À partir de nouveaux besoins de gestion, on doit choisir le bon type de trigger (BEFORE ou AFTER) et écrire la fonction qui va avec. Dans la fonction on utilise la variable **NEW** (la nouvelle ligne).

Toutes les fonctions et les triggers ont été écrits et testés dans **pgAdmin 4** avec **PostgreSQL 18**. Les scripts SQL sont dans le dossier SQL disponible sur le portfolio.

Mise en place de la base HHB-Direct

La base HHB-Direct est fournie sous la forme du script **hhb_direct_v1.sql**. On restaure cette base dans PostgreSQL sous le nom **hhbdirect** (le serveur est enregistré dans pgAdmin). Une fois restaurée, la base contient bien ses **9 tables** (client, compte, operation, agence, posseder, remunerer, type_compte, date, affecter) :



On vérifie rapidement les données, par exemple la table **client** qui contient 5 clients :

	num_client [PK] integer	nom_client character varying (30)	prenom_client character varying (30)	adresse_client character varying (192)	identifiant_internet character varying (30)	mdp_internet character varying (30)
1	1	DUPONT	Pierre	5 Rue du Port, 22300 LANNION	Pdupont	Pdupont
2	2	DUPONT	Annie	5 Rue du Port, 22300 LANNION	Adupont	Adupont
3	3	DELAVAL	Jean	12 Bd de l'Orne, 14234 OUISTREHAM	Jdelaval	Jdelaval
4	4	HANOT	Eric	13 Avenue de Neuilly, 94230 NOGENT SUR MAR...	Ehanot	Ehanot
5	5	LEVY	Sarah	1 Rue Neuve, 14110 CONDE SUR NOIREAU	Slevy	Slevy

Exercice d'application : numéro d'agence automatique

On commence par l'exercice d'application de la fiche de cours, plus simple, pour se faire la main. Insertion d'une nouvelle agence : l'utilisateur ne donne que le nom et l'adresse, le numéro d'agence doit être calculé automatiquement.

On veut calculer le numéro avant l'insertion, donc **BEFORE INSERT**. Si le numéro n'est pas fourni (**NEW.num_agence IS NULL**), on prend le plus grand numéro existant + 1.

```
1 CREATE OR REPLACE FUNCTION Insert_agence() RETURNS trigger
2 LANGUAGE plpgsql
3 AS $function$
4     begin
5         if NEW.num_agence is null then
6             select coalesce(max(num_agence), 0) + 1 into NEW.num_agence from agence ;
7         end if ;
8         return NEW ;
9     end ;
10 $function$ ;
11
12 CREATE TRIGGER Trig_insert_agence BEFORE INSERT
13 ON agence FOR EACH ROW
14 EXECUTE PROCEDURE Insert_agence() ;
```

On insère une agence à Brest **sans** donner de numéro :

```
1 INSERT INTO agence(nom_agence, adresse_agence, tel_agence)
2 VALUES ('Brest', '10 Rue de Siam, 29200 BREST', '0298001122') ;
3
4 SELECT num_agence, nom_agence, adresse_agence FROM agence ORDER BY num_agence ;
```

	num_agence [PK] integer	nom_agence character varying (50)	adresse_agence character varying (192)
1	1	CAEN Centre	13 Rue St Pierre, 14000 CAEN
2	2	Nogent sur Marne	12 Bd de Strasbourg, 94230 Nogent sur Ma...
3	3	Lannion	2 bis Rue de Brelevenez, 22300 LANNION
4	4	Brest	10 Rue de Siam, 29200 BREST

Elle a bien reçu le numéro **4** automatiquement (il y avait déjà les agences 1, 2 et 3).

Exercices

On passe maintenant aux exercices du TD. Pour chacun, on choisit d'abord le type de trigger (BEFORE ou AFTER) selon le besoin, puis on écrit la fonction associée et le déclencheur, et enfin on teste.

Exercice 1 : mise à jour des soldes lors d'un virement

Écrire le trigger qui, lors de l'insertion d'une opération de type virement, met à jour automatiquement le solde du compte source et du compte destination.

La fonction **virement()** (reprise du TP hhb-direct) insère une opération de type **VIREMENT** après avoir vérifié que les deux comptes existent et que le compte débité reste créditeur. Par contre, elle ne met ***pas*** à jour les soldes : c'est justement le rôle du trigger. Comme on veut agir **une fois** que l'opération est inscrite, on choisit un trigger **AFTER INSERT**. Voici d'abord la fonction virement() :

```
1 CREATE OR REPLACE FUNCTION virement(  
2     p_num_compte_de integer, p_id_type_de integer,  
3     p_num_compte_vers integer, p_id_type_vers integer,  
4     p_montant integer, p_designation varchar)  
5 RETURNS integer  
6 LANGUAGE plpgsql  
7 AS $function$  
8     declare  
9         solde_de integer ;  
10        existe_de integer ;  
11        existe_vers integer ;  
12        new_id bigint ;  
13    begin  
14        existe_de := (select count(*) from compte  
15                      where num_compte = p_num_compte_de and id_type = p_id_type_de) ;  
16        existe_vers := (select count(*) from compte  
17                        where num_compte = p_num_compte_vers and id_type = p_id_type_vers) ;  
18        if existe_de = 0 or existe_vers = 0 then  
19            return 0 ;  
20        end if ;  
21        select solde into solde_de from compte  
22              where num_compte = p_num_compte_de and id_type = p_id_type_de ;  
23        if solde_de - p_montant < 0 then  
24            return 0 ;  
25        end if ;  
26        if not exists (select 1 from date where date = current_date) then  
27            insert into date values (current_date) ;  
28        end if ;  
29        select coalesce(max(id_operation), 0) + 1 into new_id from operation ;  
30        insert into operation(id_operation, num_compte, date, id_type, id_type_vers,  
31                              num_compte_vers, designation, type_operation, montant)  
32        values (new_id, p_num_compte_de, current_date, p_id_type_de, p_id_type_vers,  
33              p_num_compte_vers, p_designation, 'VIREMENT', p_montant) ;  
34        return 1 ;  
35    end ;  
36 $function$
```

Petite précision : la table operation a une clé étrangère vers la table **date** (le calendrier des dates valides), donc la fonction ajoute la date du jour dans cette table si elle n'y est pas encore, sinon l'insertion serait refusée.

Ensuite la fonction du trigger et le déclencheur AFTER INSERT. On agit **uniquement** pour les opérations de type « VIREMENT » : on retire le montant au compte source et on l'ajoute au compte destination.

```

1 CREATE OR REPLACE FUNCTION Insert_operation_virement() RETURNS trigger
2 LANGUAGE plpgsql
3 AS $function$
4     begin
5         if NEW.type_operation = 'VIREMENT' then
6             update compte set solde = solde - NEW.montant
7             where num_compte = NEW.num_compte and id_type = NEW.id_type ;
8             update compte set solde = solde + NEW.montant
9             where num_compte = NEW.num_compte_vers and id_type = NEW.id_type_vers ;
10        end if ;
11        return null ;
12    end ;
13 $function$ ;
14
15 CREATE TRIGGER Trig_insert_operation_virement AFTER INSERT
16 ON operation FOR EACH ROW
17 EXECUTE PROCEDURE Insert_operation_virement() ;

```

On teste. **Avant** le virement, le compte (1,1) a un solde de 1000 et le compte (3,1) un solde de 3400 :

	num_compte [PK] integer	id_type [PK] smallint	solde integer
1	1	1	1000
2	3	1	3400

On lance un virement de 200 du compte 1 vers le compte 3. La fonction renvoie **1**, le virement a réussi :

```

1 SELECT virement(1, 1, 3, 1, 200, 'Virement test TD8') AS resultat ;

```

	resultat integer
1	1

Et les soldes sont bien mis à jour automatiquement par le trigger : **800** pour le compte source et **3600** pour le compte destination.

	num_compte [PK] integer ↗	id_type [PK] smallint ↗	solde integer ↗
1	1	1	800
2	3	1	3600

Exercice 2 : complétion automatique d'un nouveau client

Écrire le trigger qui, lors de l'insertion d'un nouveau client, détermine automatiquement son identifiant et son mot de passe internet, et crée son compte de connexion à PostgreSQL.

Ici on veut compléter la ligne **avant** qu'elle soit insérée (pour remplir l'identifiant), donc c'est un trigger **BEFORE INSERT**. On réutilise la fonction **creer_id_internet()** faite au TD précédent, qui construit l'identifiant (1ère lettre du prénom en majuscule + nom en minuscule, avec un numéro en cas d'homonymie) :

```
1 CREATE OR REPLACE FUNCTION creer_id_internet(p_nom varchar, p_prenom varchar)
2 RETURNS varchar
3 LANGUAGE plpgsql
4 AS $function$
5     declare
6         id_internet varchar ;
7         nb integer ;
8     begin
9         id_internet := upper(substr(p_prenom, 1, 1)) || lower(p_nom) ;
10        select count(*) into nb
11        from client
12        where upper(nom_client) = upper(p_nom)
13        and upper(prenom_client) = upper(p_prenom) ;
14        if nb > 0 then
15            id_internet := id_internet || nb ;
16        end if ;
17        return id_internet ;
18    end ;
19 $function$ ;
```

La fonction du trigger : si l'identifiant n'est pas saisi, on le calcule et on remplit **NEW.identifiant_internet** et **NEW.mdp_internet**. Puis on crée le rôle de connexion PostgreSQL du client (CREATE ROLE ... LOGIN).

```

1 CREATE OR REPLACE FUNCTION Insert_nouveau_client() RETURNS trigger
2 LANGUAGE plpgsql
3 AS $function$
4     declare
5         id varchar ;
6     begin
7         if NEW.identifiant_internet is null then
8             id := creer_id_internet(NEW.nom_client, NEW.prenom_client) ;
9             NEW.identifiant_internet := id ;
10            NEW.mdp_internet := id ;
11        end if ;
12        if not exists (select 1 from pg_roles where rolname = NEW.identifiant_internet) then
13            execute format('CREATE ROLE %I LOGIN PASSWORD %L',
14                NEW.identifiant_internet, NEW.mdp_internet) ;
15        end if ;
16        return NEW ;
17    end ;
18 $function$ ;
19
20 CREATE TRIGGER Trig_insert_nouveau_client BEFORE INSERT
21 ON client FOR EACH ROW
22 EXECUTE PROCEDURE Insert_nouveau_client() ;

```

On insère un nouveau client en ne donnant que son nom, son prénom et son adresse :

```

1 INSERT INTO client(num_client, nom_client, prenom_client, adresse_client)
2 VALUES (6, 'MARTIN', 'Julie', '8 Rue des Lilas, 22300 LANNION') ;
3
4 SELECT num_client, nom_client, prenom_client, identifiant_internet, mdp_internet
5 FROM client WHERE num_client = 6 ;

```

	num_client [PK] integer	nom_client character varying (30)	prenom_client character varying (30)	identifiant_internet character varying (30)	mdp_internet character varying (30)
1	6	MARTIN	Julie	Jmartin	Jmartin

L'identifiant et le mot de passe (**Jmartin**) ont bien été générés tout seuls. Et le compte de connexion PostgreSQL correspondant a été créé :

	rolname name	rolcanlogin boolean
1	Jmartin	true

Exercice 3 : contrôle des périodes de rémunération

Écrire le trigger qui, lors de l'insertion d'une nouvelle rémunération, vérifie que la nouvelle période ne chevauche pas la précédente, pour un même type de compte.

On doit pouvoir **bloquer** l'insertion si elle n'est pas valide, donc on vérifie avant : c'est un trigger **BEFORE INSERT**. On récupère la **fin** de la dernière période existante pour ce type de compte, et si le **début** de la nouvelle période n'est pas postérieur à cette fin, on lève une erreur (**RAISE EXCEPTION**) qui annule l'insertion.

```
1 CREATE OR REPLACE FUNCTION Insert_remuneration() RETURNS trigger
2 LANGUAGE plpgsql
3 AS $function$
4     declare
5         fin_precedente date ;
6     begin
7         select max(date) into fin_precedente
8         from remunerer
9         where id_type = NEW.id_type ;
10        if fin_precedente is not null and NEW.date_de <= fin_precedente then
11            raise exception 'Chevauchement de periode pour le type de compte % : le debut % do
12                NEW.id_type, NEW.date_de, fin_precedente ;
13        end if ;
14        return NEW ;
15    end ;
16 $function$ ;
17
18 CREATE TRIGGER Trig_insert_remuneration BEFORE INSERT
19 ON remunerer FOR EACH ROW
20 EXECUTE PROCEDURE Insert_remuneration() ;
```

On ajoute une période **valide** (du 01/01/2013 au 30/06/2013, après la période précédente qui finissait le 31/12/2012). L'insertion passe :

	date_de [PK] date	date [PK] date	id_type [PK] smallint	taux_interet integer
1	2012-01-...	2012-11-...	2	2
2	2012-11-...	2012-12-...	2	4
3	2013-01-...	2013-06-...	2	3

Par contre, si on essaie d'insérer une période qui **chevauche** une période déjà présente (début le 15/12/2012), le trigger **refuse** l'insertion avec notre message d'erreur :

```
ERROR: Chevauchement de periode pour le type de compte 2 : le debut 2012-12-15 doit etre posterieur a la fin de la periode precedente (2013-06-30)
CONTEXT: PL/pgSQL function insert_remuneration() line 9 at RAISE
ERREUR: Chevauchement de periode pour le type de compte 2 : le debut 2012-12-15 doit etre posterieur a la fin de la periode precedente (2013-06-30)
SQL state: P0001
```

Conclusion

Ce TD nous a fait mettre en pratique les triggers en PL/pgSQL. Le plus important, c'est de bien choisir entre **BEFORE** et **AFTER** selon le besoin : BEFORE quand on veut compléter ou vérifier/bloquer une ligne avant son insertion (exercices 2, 3 et l'exercice d'application), AFTER quand on veut réagir une fois l'opération faite (exercice 1, mise à jour des soldes). On a aussi utilisé la variable **NEW** et **RAISE EXCEPTION** pour refuser une insertion. Les triggers permettent d'automatiser des règles de gestion et de renforcer l'intégrité des données sans que l'utilisateur ait à y penser.